

UIUC CS 374

Introduction to Algorithms and Models of Computation

Anirudh Konidala

August 2026 – December 2026 (Fall 2026)

Contents

1	Lecture 01 - Introduction to CS 374	4
1.1	Alphabets	4
1.2	Strings	4
1.2.1	Example	4
1.3	Functions	4
1.3.1	Example	4
1.4	Concatenation	4
1.4.1	Example	5
1.5	Induction	5
1.5.1	Example 1	5
1.5.2	Example 2	5
1.5.3	Example 3	6
2	Lecture 10 - Recursion: Tower of Hanoi and Mergesort	6
2.1	The RealRAM Model	6
2.2	Describing Algorithms	6
2.3	Reductions and Recursion	7
2.3.1	Example: sum of an array	7
2.3.2	The recursion fairy	8
2.4	Tower of Hanoi	8
2.4.1	The recursive idea	8
2.4.2	Algorithm	9
2.4.3	Counting moves	9
2.4.4	Proof by induction	9
2.5	Mergesort	9
2.5.1	Merging	10
2.5.2	Recurrence	11
2.6	Recursion Trees	11
2.6.1	Mergesort	11
2.6.2	The three common shapes	12
3	Lecture 11 - Divide and Conquer: Quicksort and Linear Time Selection	12
3.1	Divide and Conquer	12
3.2	Quicksort	13
3.2.1	Rank	13
3.2.2	Partition	13
3.2.3	Worst case	13
3.2.4	Worst-case analysis	14
3.2.5	Good pivots	14
3.3	Selection	15

3.3.1	Sort first	15
3.3.2	Recursive idea	15
3.3.3	The selection problem	16
3.4	Quickselect	16
3.4.1	Running time	16
3.5	Median of Medians Selection	17
3.5.1	Analysis	17
3.5.2	Why blocks of 5?	18
3.5.3	Takeaways	19
3.6	Preview: Fast Multiplication	19
A	Labs	20
A.1	Lab 1A: String Induction	20
A.2	Lab 1B: Regular Expressions	22
B	Problem Sets	24
B.1	Problem Set 1	24
B.1.1	Problem 1	24

1 Lecture 01 - Introduction to CS 374

1.1 Alphabets

Let Σ be an **alphabet**, an arbitrary **finite** set of **symbols** or **characters**

Examples include:

- $\Sigma = \{A, B, \dots, Z\}$
- $\Sigma = \{0, 1\}$

1.2 Strings

A **string** or **word** is a **finite** sequence of symbols from an alphabet

Formally speaking, w is a string over Σ if it is one of

- the **empty string**
- an ordered pair (a, x) , where $a \in \Sigma$ and x is a string over Σ

1.2.1 Example

$$\begin{aligned} \text{STRING} &= (S, (\text{TRING})) \\ &= (S, (T, (\text{RING}))) \\ &= \dots \\ &= (S, (T, (R, (I, (N, (G, \varepsilon))))) \end{aligned}$$

1.3 Functions

Functions are defined recursively, where length $|w|$ represents the number of symbols in w

$$|w| := \begin{cases} 0 & \text{if } w = \varepsilon \\ 1 + |x| & \text{if } w = ax \end{cases}$$

1.3.1 Example

$$\begin{aligned} |HAT| &= 1 + |AT| \\ &= 1 + (1 + |T|) \\ &= 1 + (1 + (1 + |\varepsilon|)) \\ &= \boxed{3} \end{aligned}$$

1.4 Concatenation

Let **concatenation**, $w \bullet z$, of strings w and z be represented as

$$w \bullet z := \begin{cases} z & \text{if } w = \varepsilon \\ (a, x \bullet z) & \text{if } w = ax \end{cases}$$

1.4.1 Example

$$\begin{aligned} NOW \bullet HERE &= N \cdot (OW \bullet HERE) \\ &= N \cdot (O \cdot (W \bullet HERE)) \\ &= N \cdot (O \cdot (W \cdot (\varepsilon \bullet HERE))) \\ &= N \cdot (O \cdot (W \cdot HERE)) \\ &= \boxed{NOWHERE} \end{aligned}$$

1.5 Induction

- Consider an arbitrary object (probably of size n)
- **Explicitly** state a **strong** inductive hypothesis (i.e. for all $k < n$)
- Explicitly handle each case for the object
- Use inductive hypothesis when it's useful
- End by pointing out you handled all cases

1.5.1 Example 1

For every string w ,

$$w \bullet \varepsilon = w$$

Let w be an arbitrary string. Assume for all strings x with $|x| < |w|$, that $x \bullet \varepsilon = x$

Suppose $w = \varepsilon$. Then,

$$\begin{aligned} w \bullet \varepsilon &= \varepsilon \bullet \varepsilon && \text{since } w = \varepsilon \\ &= \varepsilon && \text{by definition of concatenation } (\bullet) \\ &= w && \text{since } w = \varepsilon \end{aligned}$$

1.5.2 Example 2

Suppose $w = a \bullet x$ for symbol a and string x . Prove that in all cases, $w \bullet \varepsilon = w$

$$\begin{aligned} w \bullet \varepsilon &= (a \cdot x) \bullet \varepsilon && (w = ax) \\ &= a \cdot (x \bullet \varepsilon) && \text{(by definition of } \bullet \text{)} \\ &= a \cdot x && \text{(inductive hypothesis)} \\ &= w && (w = ax) \end{aligned}$$

Therefore, in all cases, $w \bullet \varepsilon = w$

1.5.3 Example 3

Let w and z be arbitrary, upon the assumption that for every string x such that $|x| < |w|$, that $|x \bullet z| = |x| + |z|$

1. Suppose that $w = \varepsilon$. Then,

$$\begin{aligned} |w \bullet z| &= |\varepsilon \bullet z| & w &= \varepsilon \\ &= |z| & \text{by definition of concatenation } \bullet \\ &= 0 + |z| & \text{arithmetic} \\ &= |\varepsilon| + |z| & \text{by definition of length of string} \\ &= |w| + |z| & w &= \varepsilon \end{aligned}$$

2. Suppose that $w = a \cdot x$ for some symbol a and string x . Then,

$$\begin{aligned} |w \bullet z| &= |(a \cdot x) \bullet z| & w &= a \cdot x \\ &= |a \cdot (x \bullet z)| & \text{by definition of concatenation } \bullet \\ &= 1 + |x \bullet z| & \text{inductive hypothesis} \\ &= 1 + |x| + |z| & \text{by definition of length of string} \\ &= |w| + |z| & w &= a \cdot x \end{aligned}$$

2 Lecture 10 - Recursion: Tower of Hanoi and Mergesort

Key idea Shrink, trust, combine. Turn a big problem into smaller copies of the same problem. Trust that the copies get solved. Combine the answers.

The course now moves from *models of computation* (DFAs, grammars, Turing machines) to *algorithms*: how to solve problems **correctly** and **fast**.

2.1 The RealRAM Model

Same cost model as CS 225: anything a C-like language does cheaply is $O(1)$.

Operation	Cost
Arithmetic on a number	$O(1)$
Read or write $A[i]$	$O(1)$
Follow a pointer / reference	$O(1)$
Call a function (overhead)	$O(1)$

2.2 Describing Algorithms

What a “describe an algorithm” answer needs on homework and exams:

Give ✓	Don't give ×
Clear English or pseudocode	Real code (C++, Rust, Perl, ...)
What each function returns	A correctness proof (if you follow a class pattern)
Running time: recurrence + solution	A loose bound “just to be safe”

Watch out Greedy algorithms need a formal proof. They often look right and are wrong. Without a proof, graders assume a greedy algorithm is incorrect.

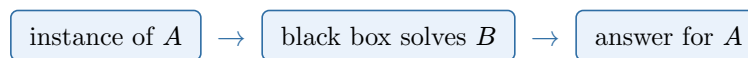
How answers are graded (target runtimes are not given):

Correct?	Speed	Credit
Yes	Faster than expected	Extra credit
Yes	Slower than expected	Big partial credit
No	Fast	Very little

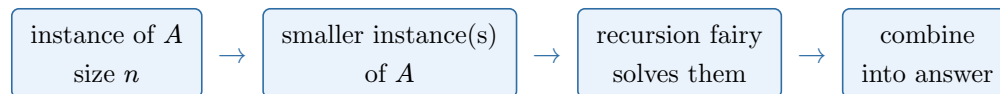
Key idea Correctness first, then speed.

2.3 Reductions and Recursion

A **reduction** solves problem A by using a solver for problem B as a **black box** (you never look inside it):



Recursion is a reduction where B is a **smaller copy of A itself**:



If the instance is too small to shrink, answer it directly. That is the **base case**.

2.3.1 Example: sum of an array

```

Sum(A[1..n]):
  if n = 0
    return 0                ((base case))
  return Sum(A[1..n - 1]) + A[n]  ((trust the smaller call))
  
```

$$\text{Sum}(\boxed{3} \mid \boxed{1} \mid \boxed{4}) = \text{Sum}(\boxed{3} \mid \boxed{1}) + 4 = 4 + 4 = 8$$

Blue part: handed to the recursion fairy, who returns 4. Yellow part: our own work.

2.3.2 The recursion fairy

Key idea Never trace what happens **inside** a recursive call. Assume it returns the right answer. Tracing the calls of the calls overflows your brain.

Why trusting is allowed: **induction** (Lecture 01). Assume the algorithm is correct on every smaller input (inductive hypothesis). Then show it is correct on size n .

Your three jobs:

1. Every recursive call is on a **strictly smaller** instance of the **same** problem
2. Every **base case** is answered correctly
3. The **combine step** is correct, assuming the recursive calls were correct

2.4 Tower of Hanoi

Puzzle by Édouard Lucas (1883). Move the whole stack from **src** to **dst**.

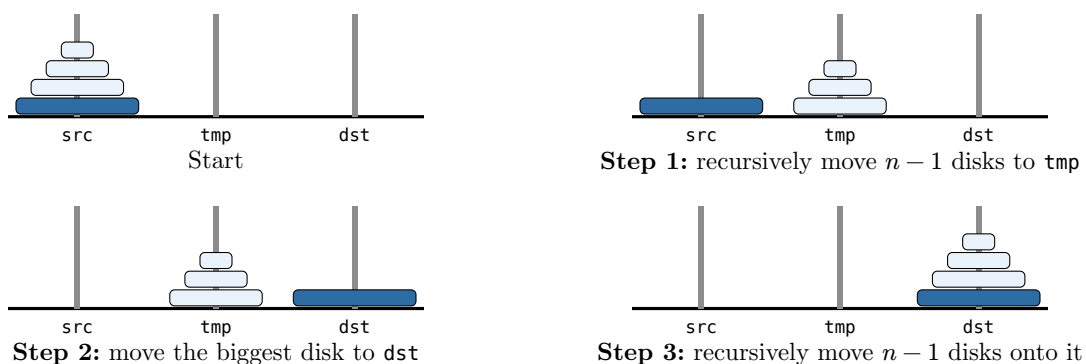


Rules:

- Move **one** disk at a time, from the top of a pile to the top of another peg
- **Never** put a bigger disk on a smaller one

2.4.1 The recursive idea

The biggest disk (dark) must move to **dst** at some point. To move it, every smaller disk must be out of the way, stacked on **tmp**.



Key idea The big disk never gets in the way: any disk may sit on it. So moving the $n - 1$ small disks is **the same puzzle**, just smaller, with a different target peg.

2.4.2 Algorithm

State what the call does, so a reader can check it:

```
Hanoi(n, src, dst, tmp):  ((move disks 1..n from src to dst))
  if n > 0
    Hanoi(n - 1, src, tmp, dst)    ((step 1))
    move disk n from src to dst    ((step 2))
    Hanoi(n - 1, tmp, dst, src)    ((step 3))
```

Base case: $n = 0$, nothing to move.

2.4.3 Counting moves

Let $T(n)$ be the number of moves for n disks. Two recursive calls, plus one move:

$$T(n) = \begin{cases} 0 & \text{if } n = 0 \\ 2T(n-1) + 1 & \text{if } n > 0 \end{cases}$$

This is a **recurrence**: the cost is written in terms of the cost of smaller inputs. Compute small values and spot the pattern:

n	0	1	2	3	4	5
$T(n)$	0	1	3	7	15	31
$2^n - 1$	0	1	3	7	15	31

$$\boxed{T(n) = 2^n - 1}$$

2.4.4 Proof by induction

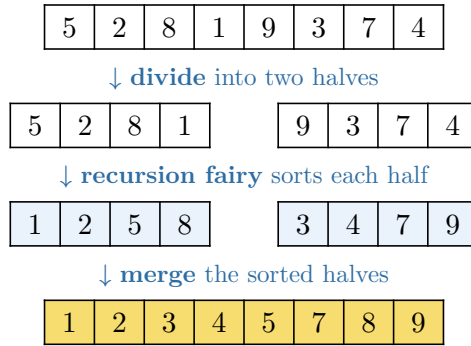
Guess-and-verify: check the guess with induction. Assume $T(k) = 2^k - 1$ for all $k < n$.

- $n = 0$: $T(0) = 0 = 2^0 - 1$ ✓
- $n > 0$:

$$\begin{aligned} T(n) &= 2T(n-1) + 1 && \text{definition of } T \\ &= 2(2^{n-1} - 1) + 1 && \text{inductive hypothesis} \\ &= 2^n - 1 && \text{arithmetic} \end{aligned}$$

2.5 Mergesort

Sort $A[1..n]$ into increasing order. By von Neumann (1945).



```

MergeSort(A[1..n]):
  if n > 1
    m ← ⌊n/2⌋
    MergeSort(A[1..m])
    MergeSort(A[m+1..n])
    Merge(A[1..n], m)

```

Base case: $n \leq 1$. Already sorted, nothing to do.

2.5.1 Merging

Both halves are sorted, so the smallest remaining element is always at the **front** of one half. Repeatedly compare the two fronts and take the smaller one:

Left half	Right half	Take	Output so far
1258	3479	1	1
258	3479	2	1 2
58	3479	3	1 2 3
58	479	4	1 2 3 4
58	79	5	1 2 3 4 5
⋮	⋮	⋮	⋮

Each comparison places one element, so Merge takes $O(n)$ time. One way to write it:

```

Merge(A[1..n], m):
  i ← 1; j ← m + 1      ((fronts of the two halves))
  for k ← 1 to n
    if j > n              ((right half used up))
      B[k] ← A[i]; i ← i + 1
    else if i > m          ((left half used up))
      B[k] ← A[j]; j ← j + 1
    else if A[i] < A[j]
      B[k] ← A[i]; i ← i + 1
    else
      B[k] ← A[j]; j ← j + 1

```

```

    B[k] ← A[j];  j ← j + 1
  for k ← 1 to n
    A[k] ← B[k]

```

2.5.2 Recurrence

Two half-size recursive calls, plus $O(n)$ for the merge:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Key idea Floors and ceilings ($\lfloor \frac{n}{2} \rfloor$ vs $\lceil \frac{n}{2} \rceil$) don't matter when the problem shrinks by a constant fraction. Just write $\frac{n}{2}$.

2.6 Recursion Trees

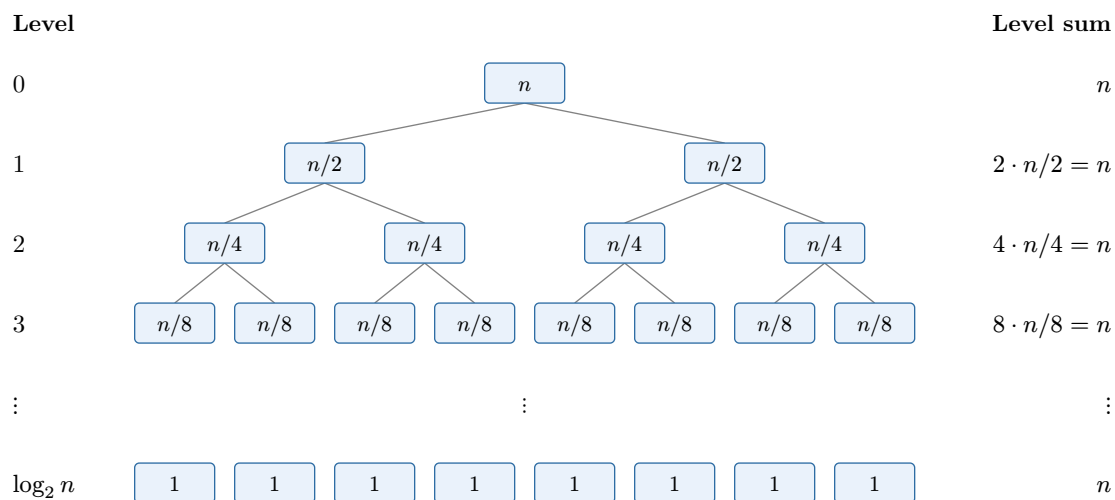
A picture of every recursive call, used to **solve** recurrences.

- One **node** per recursive call. The **root** is the first call.
- A node's **children** are the recursive calls it makes.
- A node's **value** is the work done in that call, **not counting** its children.

Key idea **Total time = sum of all node values.** Add up each level (row) first, then add up the levels.

2.6.1 Mergesort

The root does n work (the merge). Each child has half the elements, so it does $\frac{n}{2}$ work. Each grandchild does $\frac{n}{4}$. And so on, down to size 1.



- **Every level sums to n .**

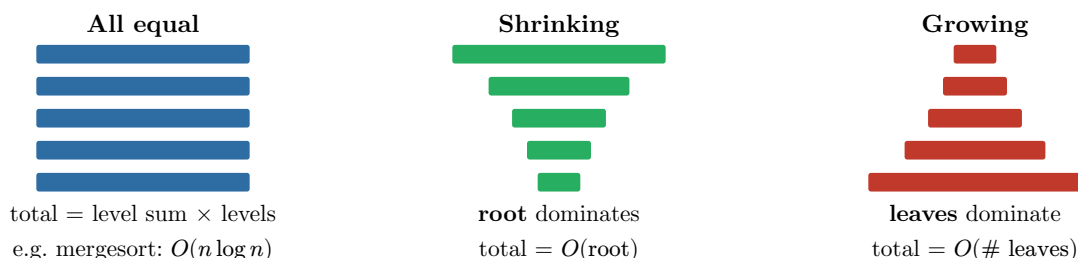
- **How many levels?** Each level halves the size: $n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \dots \rightarrow 1$. You can halve n only $\log_2 n$ times before reaching 1.

$$T(n) = \underbrace{n}_{\text{per level}} \times \underbrace{\log_2 n}_{\text{levels}} = \boxed{O(n \log n)}$$

The log base doesn't matter inside big- O (changing base is a constant factor).

2.6.2 The three common shapes

Look at how the level sums change from top to bottom:



Shrinking and growing are **geometric series**: each level is the one above times a fixed ratio $\neq 1$ (for example $n, \frac{n}{3}, \frac{n}{9}, \dots$). A geometric series is proportional to its **biggest term**, however many terms it has. Lecture 11 has examples of both.

Watch out Don't memorize a formula theorem instead. It only fits recurrences with equal-size pieces. Lecture 11's recurrences don't fit it, but recursion trees still work.

3 Lecture 11 - Divide and Conquer: Quicksort and Linear Time Selection

3.1 Divide and Conquer

Mergesort is an example of a **divide and conquer** algorithm. There are no hard rules for what counts as divide and conquer, but the high-level idea is:

1. **Divide** the problem into smaller instances of the same problem
2. **Delegate** the smaller instances to the recursion fairy
3. **Combine** the solutions of the subproblems into a solution for the original instance. This step usually needs some extra work.

For mergesort: divide into two halves of size about $\frac{n}{2}$, sort both recursively, then combine with **Merge**. Combining two sorted arrays is much easier than sorting from scratch.

To analyze a divide and conquer algorithm, write a **recurrence** for the running time in terms of the smaller instances, then solve it (usually with a recursion tree).

3.2 Quicksort

Tony Hoare (1961).

1. Choose a **pivot** element. Any element is allowed; the choice is arbitrary.
2. **Partition** the array in linear time: everything smaller than the pivot moves to its left, everything bigger moves to its right.
3. Now the pivot is in its final position, but the two sides are not sorted. Neither side contains the pivot, so both are strictly smaller. Recursively sort the left side, then recursively sort the right side.

```
QuickSort(A[1..n]):  
  if n > 1  
    choose a pivot element A[p]  
    r ← Partition(A, p)  
    QuickSort(A[1..r - 1])  
    QuickSort(A[r + 1..n])
```

3.2.1 Rank

Partition returns r , the new index of the pivot. This is the **rank** of the pivot:

- $\text{rank}(x) = 1 + (\text{number of elements smaller than } x)$
- Equivalently, the position of x in sorted order (1-indexed)

After partitioning, the elements smaller than the pivot are in $A[1..r - 1]$ and the elements larger are in $A[r + 1..n]$. Duplicate elements cause no problem: take the rank to mean the pivot's final position.

3.2.2 Partition

For completeness only. The correctness argument is subtle; see the lecture notes and Erickson's book. Move the pivot to the end, out of the way. Walk through the array; whenever an element is smaller than the pivot, swap it into the left part. Finally, swap the pivot into place.

```
Partition(A[1..n], p):  
  swap A[p] ↔ A[n]  
  ℓ ← 0                                ((#items < pivot))  
  for i ← 1 to n - 1  
    if A[i] < A[n]  
      ℓ ← ℓ + 1  
      swap A[ℓ] ↔ A[i]  
  swap A[n] ↔ A[ℓ + 1]  
  return ℓ + 1
```

Partitioning takes $O(n)$ time.

3.2.3 Worst case

The recursive calls do **not** have to be on halves. With pivot rank r :

$$T(n) = T(r - 1) + T(n - r) + O(n)$$

If the pivot is the smallest (or largest) element, one call is empty and the other has $n - 1$ elements:

$$T(n) = T(n - 1) + O(n)$$

The recursion tree is a long path:

Level	Nodes	Sum
0	$\boxed{n}$	n
1	$\boxed{n - 1}$	$n - 1$
2	$\boxed{n - 2}$	$n - 2$
$\vdots$	$\boxed{\dots}$	$\vdots$
$n - 1$	$\boxed{1}$	1

This is an **arithmetic series**, not a geometric one. Its sum is **not** proportional to the largest term. It is proportional to (largest term) $\times$ (number of terms):

$$T(n) = \sum_{i=1}^n i = \boxed{O(n^2)}$$

Every simple deterministic pivot rule has bad inputs that force this: the first element, the last element, even the median of the first, middle and last elements.

3.2.4 Worst-case analysis

This course uses worst-case analysis. A stated running time must hold for every input. So all we can promise for quicksort is $O(n^2)$.

A loose bound such as $O(n!)$ is technically correct but useless, and is marked wrong. Give the bound that naturally falls out of your analysis technique.

3.2.5 Good pivots

Quicksort is fast in practice: unless someone gives it nasty inputs on purpose, the pivot rank is usually near $\frac{n}{2}$. With probability about $\frac{1}{3}$, the pivot is in the **middle third**, so both calls have at most $2n/3$ elements. In the most unbalanced such case:

$$T(n) \leq T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + O(n)$$

Level	Nodes	Sum
0	$\boxed{n}$	n
1	$\boxed{\frac{n}{3}} \quad \boxed{\frac{2n}{3}}$	n
2	$\boxed{\frac{n}{9}} \quad \boxed{\frac{2n}{9}} \quad \boxed{\frac{2n}{9}} \quad \boxed{\frac{4n}{9}}$	n
$\vdots$	$\boxed{\dots}$	$\leq n$

The tree is unbalanced, but every level sums to (at most) n . Some branches reach their base cases early, so lower levels may be missing nodes; that is fine for an upper bound.

The longest path follows the larger calls, multiplying by $\frac{2}{3}$ each level. The number of times you can multiply n by $\frac{2}{3}$ before reaching 1 is the number of times you can divide by $\frac{3}{2}$:

$$\text{levels} \leq \log_{3/2} n = O(\log n)$$

Any constant log base changes the result only by a constant factor, so:

$$T(n) = n \cdot \log_{3/2} n = \boxed{O(n \log n)}$$

3.3 Selection

Median: given $A[1..n]$, find the element of rank $\lceil \frac{n}{2} \rceil$.

3.3.1 Sort first

Mergesort the array and return $A[\lceil \frac{n}{2} \rceil]$. This takes $O(n \log n)$ time, and is a perfectly valid answer on a homework or exam. **If you can use something from class as a black box, do so.** But we only want one element, so try to do better.

3.3.2 Recursive idea

Pick a pivot and partition. Compare its rank r with $\lceil \frac{n}{2} \rceil$:

- $r < \lceil \frac{n}{2} \rceil$: everything left of the pivot is even smaller than the median, so the median is to the **right**. Recurse on the right side.
- $r > \lceil \frac{n}{2} \rceil$: the median is to the **left**. Recurse on the left side.
- $r = \lceil \frac{n}{2} \rceil$: the pivot is the median.

Do not pick another pivot inside the right side yourself. That is doing the recursion fairy's job, and it makes them grumpy. Just recurse.

Technical issue: inside the right subarray, the median no longer has rank $\lceil \frac{n}{2} \rceil$. Its rank is smaller relative to the subarray. So the original problem is not general enough to recurse on. Sometimes you need to solve a **more general problem** that you can call recursively.

3.3.3 The selection problem

Given an array $A[1..n]$ and an integer k with $1 \leq k \leq n$, find the element of rank k .

3.4 Quickselect

Tony Hoare, on the same page of the paper as quicksort. Think of it as a **one-armed quicksort**.

QuickSelect($A[1..n]$, k):

```

    if  $n = 1$ 
        return  $A[1]$ 
    else
        choose a pivot element  $A[p]$ 
         $r \leftarrow \text{Partition}(A[1..n], p)$ 
        if  $k < r$ 
            return QuickSelect( $A[1..r - 1]$ ,  $k$ )
        else if  $k > r$ 
            return QuickSelect( $A[r + 1..n]$ ,  $k - r$ )
        else
            return  $A[r]$ 

```

- $k < r$: everything right of the pivot is too big. Search the left side, still for rank k .
- $k > r$: the target is bigger than the pivot. Search the right side. The right side is missing the r smallest elements, so look for rank $k - r$.
- $k = r$: the pivot is the answer.

3.4.1 Running time

Worst case: the pivot is always the smallest or largest element. Same path-shaped recursion tree as quicksort, so $O(n^2)$.

With a pivot in the middle third, there is only **one** recursive call, on at most $2n/3$ elements:

$$T(n) \leq T\left(\frac{2n}{3}\right) + O(n)$$

Level	Nodes	Sum
0	$\boxed{n}$	n
1	$\boxed{\frac{2n}{3}}$	$\frac{2n}{3}$
2	$\boxed{\frac{4n}{9}}$	$\frac{4n}{9}$
i	$\boxed{\left(\frac{2}{3}\right)^i n}$	$\left(\frac{2}{3}\right)^i n$

The level sums form a **decreasing geometric series**. A geometric series whose ratio is not 1 is proportional to its largest term, here the root. So $T(n) = O(n)$, **if** we can always find a pivot near the middle.

3.5 Median of Medians Selection

Blum, Floyd, Pratt, Rivest and Tarjan (1970). Idea: take a representative **sample** of the array, find the sample's median recursively, and use it as the pivot. The sample's median should be close to the real median.

```
MomSelect(A[1..n], k):
  if n ≤ 25
    use brute force
  else
    m ← ⌈n/5⌉
    for i ← 1 to m
      M[i] ← MedianOfFive(A[5i - 4..5i])
    mom ← MomSelect(M[1..m], ⌊m/2⌋)
    r ← Partition(A[1..n], mom)
    if k < r
      return MomSelect(A[1..r - 1], k)
    else if k > r
      return MomSelect(A[r + 1..n], k - r)
    else
      return mom
```

1. **Big base case:** if $n \leq 25$, sort and pick directly. Constant time.
2. Split the array into blocks of 5. Find the median of each block and put it in array M of size about $\frac{n}{5}$.
3. **Recursively** find the median of M , the **median of medians** (mom). This is allowed because $\frac{n}{5} < n$.
4. Run quickselect with mom as the pivot.

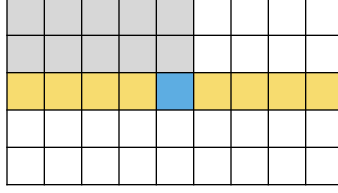
You can think of step 3 as an **approximate median fairy**, which is secretly the recursion fairy in disguise.

3.5.1 Analysis

Work outside the recursive calls: each median of five takes constant time, and there are $\frac{n}{5}$ of them; partitioning takes $O(n)$. So $O(n)$ total.

The question: is mom a good pivot? Show that neither recursive call is too big. **For the analysis only** (the algorithm does not do this), arrange A into a $5 \times \frac{n}{5}$ grid:

1. Each column is one block of 5.
2. Sort each column, smallest on top. The block medians land in the middle row.
3. Sort the columns by their medians. mom is now in the center.



block medians
 mom
 $\leq$ mom

- Each median to the left of mom is smaller than mom.
- Each column is sorted, so the elements above those medians are smaller still.
- There are $\frac{n}{5}$ columns, so about $\frac{n}{10}$ columns are left of mom. Each has 3 elements (the median and the two above it) smaller than mom.

So at least $3n/10$ elements are smaller than mom, and **at most $7n/10$ elements are larger**. Symmetrically, at most $7n/10$ elements are smaller. The other cells could go either way, but they are not needed. Either recursive call has at most $7n/10$ elements:

$$T(n) \leq O(n) + T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right)$$

Level	Nodes	Sum
0	$\boxed{n}$	n
1	$\boxed{\frac{n}{5}} \quad \boxed{\frac{7n}{10}}$	$\frac{9n}{10}$
2	$\boxed{\frac{n}{25}} \quad \boxed{\frac{7n}{50}} \quad \boxed{\frac{7n}{50}} \quad \boxed{\frac{49n}{100}}$	$\frac{81n}{100}$
i	$\boxed{\dots}$	$\left(\frac{9}{10}\right)^i n$

$\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, so the level sums form a **decreasing geometric series**. The number of levels does not matter; the sum is proportional to the root:

$$T(n) = \boxed{O(n)}$$

This is optimal: any algorithm must look at every element.

3.5.2 Why blocks of 5?

5 is the smallest odd block size for which the analysis works (7 also works). With blocks of 3, the same argument gives

$$T(n) \leq O(n) + T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right)$$

Every level sums to n , which is the good-pivot quicksort tree: $O(n \log n)$, no better than sorting.

3.5.3 Takeaways

- Divide and conquer can use **unexpected** subproblems, such as recursing on a sample just to find a pivot.
- Subproblems can be **unbalanced**. Use recursion trees to analyze them.

3.6 Preview: Fast Multiplication

Next lecture: multiply two n -digit numbers with $O(n^{\log_2 3}) \approx O(n^{1.59})$ single-digit multiplications. Another divide and conquer algorithm, which splits each number's digits into two halves.

Appendix A: Labs

A.1 Lab 1A: String Induction

The following problems ask you to prove some “obvious” claims about recursively-defined string functions. In each case, we want a self-contained, step-by-step induction proof that builds on formal definitions and prior results, *not* on intuition. In particular, your proofs must refer to the formal recursive definitions of string length and string concatenation:

$$|w| := \begin{cases} 0 & \text{if } w = \varepsilon \\ 1 + |x| & \text{if } w = ax \text{ for some symbol } a \text{ and some string } x \end{cases}$$
$$w \bullet z := \begin{cases} z & \text{if } w = \varepsilon \\ a \cdot (x \bullet z) & \text{if } w = ax \text{ for some symbol } a \text{ and some string } x \end{cases}$$

You may freely use the following results:

Lemma 1. $w \bullet \varepsilon = w$ for all strings w .

Lemma 2. $|w \bullet z| = |w| + |z|$ for all strings w and z .

Lemma 3. $(w \bullet y) \bullet z = w \bullet (y \bullet z)$ for all strings w, y , and z .

The **reversal** w^R of a string w is defined recursively as follows

$$w^R := \begin{cases} \varepsilon & \text{if } w = \varepsilon \\ x^R \bullet a & \text{if } w = ax \text{ for some symbol } a \text{ and some string } x \end{cases}$$

For example, **STRESSED**^R = **DESSERTS** and **WTF374**^R = **473FTW**.

1. Prove that $|w| = |w^R|$ for every string w .

Assume that $|x| = |x^R|$ for every string x shorter than w .

Case 1: Suppose that $w = \varepsilon$.

Then,

$$\begin{aligned} |w^R| &= |\varepsilon^R| \\ &= |\varepsilon| \text{ by definition of reversal} \\ &= |w| \end{aligned}$$

Case 2: $w = ax$ for some symbol a and string x .

Then,

$$\begin{aligned}
|w^R| &= |x^R \bullet a| && \text{by definition of reversal} \\
&= |x^R| + |a| && \text{by definition of **Lemma 2**} \\
&= |x^R| + 1 && \text{by definition of length (since } |a| = |a\varepsilon| = 1 + |\varepsilon| = 1) \\
&= |x| + 1 && \text{by definition of inductive hypothesis} \\
&= |ax| && \text{by definition of length} \\
&= |w|
\end{aligned}$$

2. Prove that $(w \bullet z)^R = z^R \bullet w^R$ for all strings w and z .

Assume that $(x \bullet z)^R = z^R \bullet x^R$ for every string x shorter than w (and every string z)

Case 1: Suppose that $w = \varepsilon$.

Then,

$$\begin{aligned}
(w \bullet z)^R &= (\varepsilon \bullet z)^R \\
&= z^R && \text{by definition of concatenation} \\
&= z^R \bullet \varepsilon && \text{by definition of **Lemma 1**} \\
&= z^R \bullet \varepsilon^R && \text{by definition of reversal} \\
&= z^R \bullet w^R
\end{aligned}$$

Case 2: Suppose that $w = ax$ for some symbol a and string x .

Then,

$$\begin{aligned}
(w \bullet z)^R &= (ax \bullet z)^R \\
&= (a \cdot (x \bullet z))^R && \text{by definition of concatenation} \\
&= (x \bullet z)^R \bullet a && \text{by definition of reversal} \\
&= (z^R \bullet x^R) \bullet a && \text{by definition of inductive hypothesis} \\
&= z^R \bullet (x^R \bullet a) && \text{by definition of **Lemma 3**} \\
&= z^R \bullet w^R && \text{by definition of reversal}
\end{aligned}$$

3. Prove that $(w^R)^R = w$ for every string w

Assume that $(x^R)^R = x$ for every string x shorter than w .

Case 1: Suppose that $w = \varepsilon$.

Then,

$$\begin{aligned}
(w^R)^R &= (\varepsilon^R)^R \\
&= \varepsilon^R && \text{by definition of reversal} \\
&= \varepsilon && \text{by definition of reversal} \\
&= w
\end{aligned}$$

Case 2: Suppose that $w = ax$ for some symbol a and string x

Then,

$$\begin{aligned}
(w^R)^R &= ((ax)^R)^R \\
&= (x^R \bullet a)^R && \text{by definition of reversal} \\
&= a^R \bullet (x^R)^R && \text{by Problem 2} \\
&= a^R \bullet x && \text{by the inductive hypothesis} \\
&= a \bullet x && \text{by definition of reversal } (a = a\varepsilon; a^R = \varepsilon^R \bullet a = \varepsilon \bullet a = a) \\
&= ax && \text{by definition of concatenation} \\
&= w
\end{aligned}$$

A.2 Lab 1B: Regular Expressions

A *regular expression* is compact notation for a language (that is, a set of strings). Formally, a regular language is one of the following:

- The symbol $\emptyset$ (representing the empty set)
- Any string (representing the set containing only that string)
- $R + S$ for some regular expressions R and S (representing alternation / union)
- $R \cdot S$ or RS for some regular expressions R and S (representing concatenation)
- R^* for some regular expression R (representing Kleene closure / unbounded repetition)

In the absence of parentheses, Kleene closure has highest precedence, followed by concatenation. For example:

$$\begin{aligned}
0 + RE^* &= 0 + R(E^*) = \{0, R, RE, REE, REEE, REEEE, REEEEE, REEEEE, REEEEE, \dots\} \\
0 + (RE)^* &= \{\varepsilon, 0, RE, RERE, RERERE, RERERERE, RERERERERE, \dots\} \\
(0 + R)E^* &= \{0, R, 0E, RE, OEE, REE, OEEE, REEE, OEEEE, REEEE, OEEEE, \dots\} \\
(0 + RE)^* &= \{\varepsilon, 0, 00, RE, 000, ORE, REO, 0000, OORE, OREO, RE00, RERE, \dots\}
\end{aligned}$$

Give regular expressions for each of the following languages over the binary alphabet $\{0, 1\}$. (For extra practice, find multiple regular expressions for each language.)

0. All strings.

$$(\textcolor{red}{0} + \textcolor{red}{1})^*$$

1. All strings containing the substring $\textcolor{red}{000}$.

$$((\textcolor{red}{0} + \textcolor{red}{1})^* \textcolor{red}{000} (\textcolor{red}{0} + \textcolor{red}{1})^*)$$

2. All strings *not* containing the substring $\textcolor{red}{000}$.

$$(\textcolor{red}{1} + \textcolor{red}{01} + \textcolor{red}{001})^*(\varepsilon + \textcolor{red}{0} + \textcolor{red}{00})$$

3. All strings in which every run of $\textcolor{red}{0}$ s has length at least 3.
4. All strings in which the *last* $\textcolor{red}{1}$ appears before the *first* substring $\textcolor{red}{000}$.
5. All strings containing at least three $\textcolor{red}{0}$ s.
6. Every string except $\textcolor{red}{000}$. [*Hint: Don't try to be clever.*]

Appendix B: Problem Sets

B.1 Problem Set 1

B.1.1 Problem 1